Counting Sort

- ordenação de base de dados. registro inicial tem que ser numero inteiro
- pode ser adaptado pra números negativos
- nao serve pra números reais, apenas inteiros

Funcionamento

- Verifica num vetor inicial a ocorrência de um número
- Pode ter números iguais
- Ordenação por ordem de ocorrência

Exemplo:

Vetor original (vet) → desordenado

10	2	10	7	8	100	3	5	n=9
0	1	2	3	4	5	6	7	

- Inicialmente: achar o maior elemento desse valor
- sempre começa com a primeira posição do vetor

Começo do código:

```
int maior = vet[0]
for(i =1;i<n;i++){
    if(vet[i]>maior)
        maior=vet[i]
}
maior=> 100

int tam = maior+1;
int *vetCount = new int[tam];
```

Em seguida: criar um for pra percorrer o vetor original (n=9) → `for(i=0,i<n,i++)`

- Ele deve: verificar o valor no vet e anotar ocorrencia (passar pro vetor vetCount)
  → o vet[i] seria a posição do vetor vetC (vetor counting)

Vetor Counting (vetCount)

- Criação do vetor auxiliar dinãmico
- OBS: o tamanho, nesse exemplo, ficará 101 pois o vetor em c++ começa no número 0.
  - se fosse tam=100 o vetor teria o último índice até 99
    - Ou seja: o maior elemento do vetor original seria o 99, não o 100;
- Inicia o vetor totalmente com zero:

0	0	0	0	0	0	0	0

0	1	2	3	4	5	6	7

- Cada posição desse vetor referencia um valor de um elemento no vetor original
- As ocorrências de cada numero no vetor original será atribuído como “elemento” do vetorCount:
  - onde tem zero→ não existe ocorrência no vetor original
  - onde tem um→ uma ocorrência (elemento presente uma vez)
  - onde tem dois→ duas ocorrências (elemento presente duas vezes) ...
- A soma dessas ocorrências tem que dar o valor do tamanho do vetor original
  - Nesse exemplo: a soma tem que dar nove

- Depois de percorrer todo o vetor original, temos a seguinte configuração

0	0	1	2	0	1	0	1
0	1	2	3	4	5	6	7

indice: j

<u>Desvantagem:</u>

→ se eu tiver o número 1000 como maior elemento do vetor: cria um vetor de 1001 posições pra ordenar apenas 9 elementos→ cria vetor muito grande na memória

quando n devo usar o counting: quando houver um espaçamento muito grande entre o menor e o maior elemento (quando não segue um padrão (ex: até 10, até 20)

<u>Otimizações</u>:

- analisar também o menor elemento
- No código: `maior-menor+1` → n precisaria criar do 0 ao 50, apenas 50 ao 100
- duas possibilidades para dizer pra nao realizar a pesquisa:
  - if(n≤0)→ se for tamanho zero
  - if(n≤1)→ já está ordenado

Porque não é indicado percorrer do inicio ao fim: pode ser q percorra do inicio ao meio e não tenha nenhuma ocorrência armazenada

- se começo do 100→ apenas preciso percorrer até quando preencher todo o vetor original

→ percorre até um elemento:

```
for(i=n-1;j=tam-1; i<=0; j--){ //aq signifca que atualizou o j
//for que diz se há ocorrencias de elementos(valor diferente de zero,como 1 ou 2)
    for(x=vetCount[j];x>0){
        vet[i]=j;
        vetCount[j]--;
```

```
        i--;
    }
}
```

j: ou tam-1, ou maior

OBS:

- A otimização desse for : percorre também de trás pra frente, inserindo no vetor original: (fim=8)
- em `vet[i]=j;` , pode sobrescrever o vetor original, já que os elementos já foram lidos

Considerando vetCount:

Pra percorrer:

- sai da posição quando o valor for 0
- se não for igual a zero, decrementa até chegar no zero (Ex: se valia um, agr vale 0), conforme vai adicionando os elementos no valor orginal

0	0	1	2	0	1	0	1
0	1	2	3	4	5	6	7

← ^

Considerando vet (original)

- só adiciona elemento quando no vetCount tiver valor ≠ de zero

2	3	3	5	7	8	10	10
0	1	2	3	4	5	6	7

j=100

j=99

...

j=10

j=9